Dat Nguyen

1. Overview of user-kernel communication interfaces: System Call, IOCTL, proc file system, Netlink Socket - what each is for and when to choose it

User Space and Kernel Space

Linux divides the execution environment into two main regions.

- User Space is where regular applications run, such as:

```
ip: configures network interfaces, IP addresses, and routing.

ss: displays socket information and network connections.

NetworkManager: manages network connections.

C programs written by you.

Applications that control devices through device drivers.
```

- Kernel Space is where the Linux Kernel performs core tasks:

```
Process Management

Memory Management

Networking

File Systems

Device Drivers
```

For example, when we run the command: "**ip link show**"

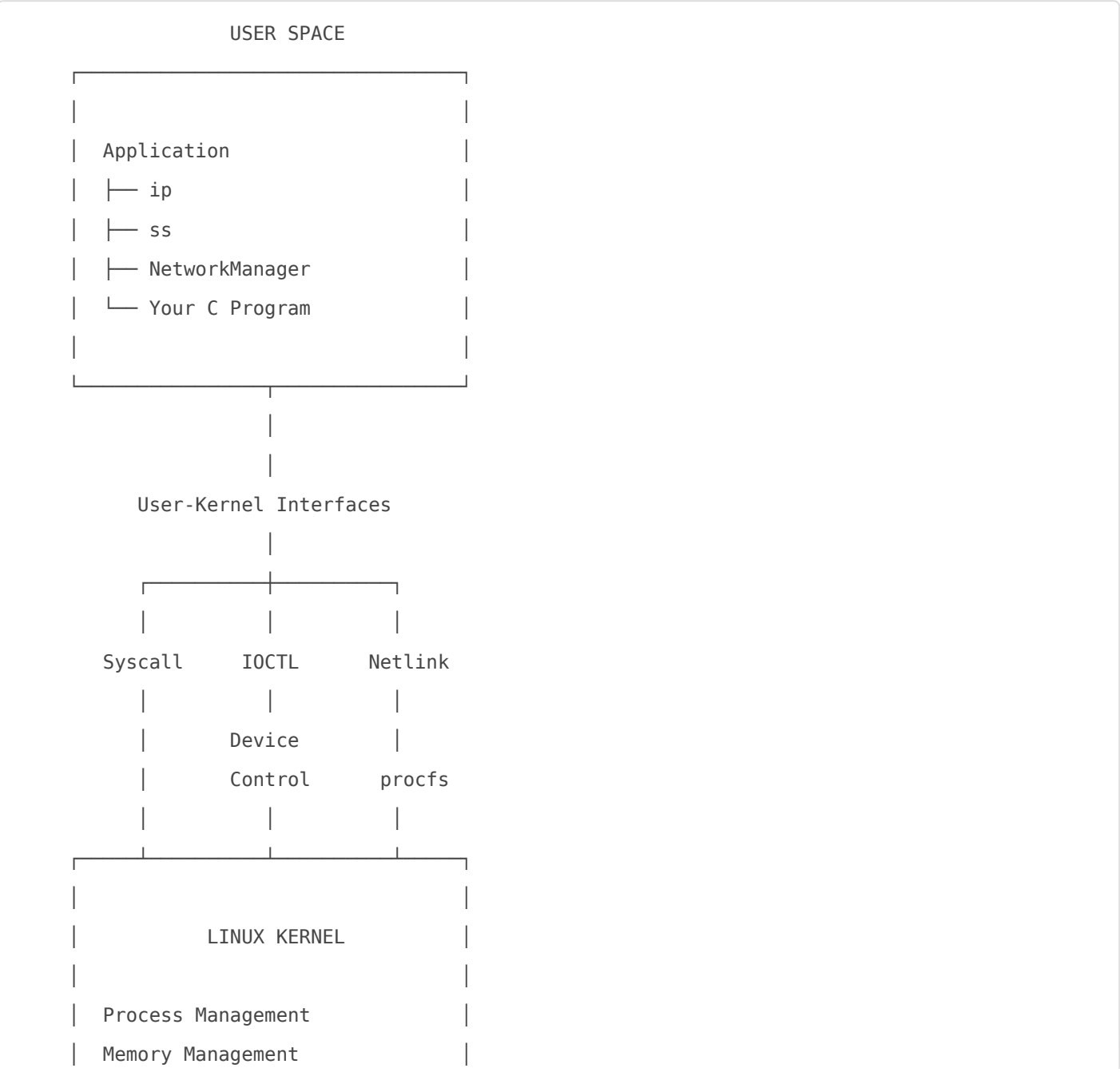
The ip program runs in User Space, but information about network interfaces like wlo1, interface status, and MTU is managed by the Linux Kernel.

So the question is:

- How does the ip program get the information managed by the Kernel? A User Space program cannot arbitrarily read or modify the Kernel's internal memory directly.
- Instead, it must use the interfaces provided by the Kernel.

=> That is the User-Kernel Communication Interfaces.

General Communication Model



```
    │  Networking Subsystem           │
    │  Device Drivers                 │
    │                                 │
    └─────────────────────────────────┘
```

Note: These four interfaces are not completely independent of each other. They can work together to form a complete communication mechanism.

Overview Comparison Table

Interface	Main Purpose	Common Usage	Examples
System Call	Ask the kernel to perform a specific task.	Call system calls from C programs.	`getpid()`, `read()`, `write()`
IOCTL	Send special control commands to a device or driver.	`ioctl(fd, request, arg)`	Turn a device on or off, change its operating mode
procfs	Provide kernel information or allow configuration through a filesystem.	Read or write files in `/proc`.	`/proc/cpuinfo`, `/proc/net/`
Netlink Socket	Exchange structured messages between User Space and Kernel Space.	`socket(AF_NETLINK, ...)`	Get network interface information, routing information

2 System Call interface: deep dive + writing a simple C program that invokes a syscall directly via syscall(2)

2.1 What is a System Call?

A System Call is an interface that allows a User Space program to request the Linux Kernel to perform operations provided by the Kernel (such as creating a new process, reading/writing files, or creating a pipe for inter-process communication).

Main characteristics:

- Triggers a CPU state switch from user mode to kernel mode to access protected kernel memory.

Kernel memory protection: The operating system's memory (kernel space) is strictly protected. Normal applications cannot casually poke around in it to prevent system crashes or security breaches by malicious software. When a system call is invoked, the CPU pauses your program,

switches to the privileged "kernel mode" to execute the request, and then safely returns control back to your program.

- The set of system calls is fixed; each has a distinct number, but when programming, we refer to them by name.
- Parameters can be passed between user space and kernel space.

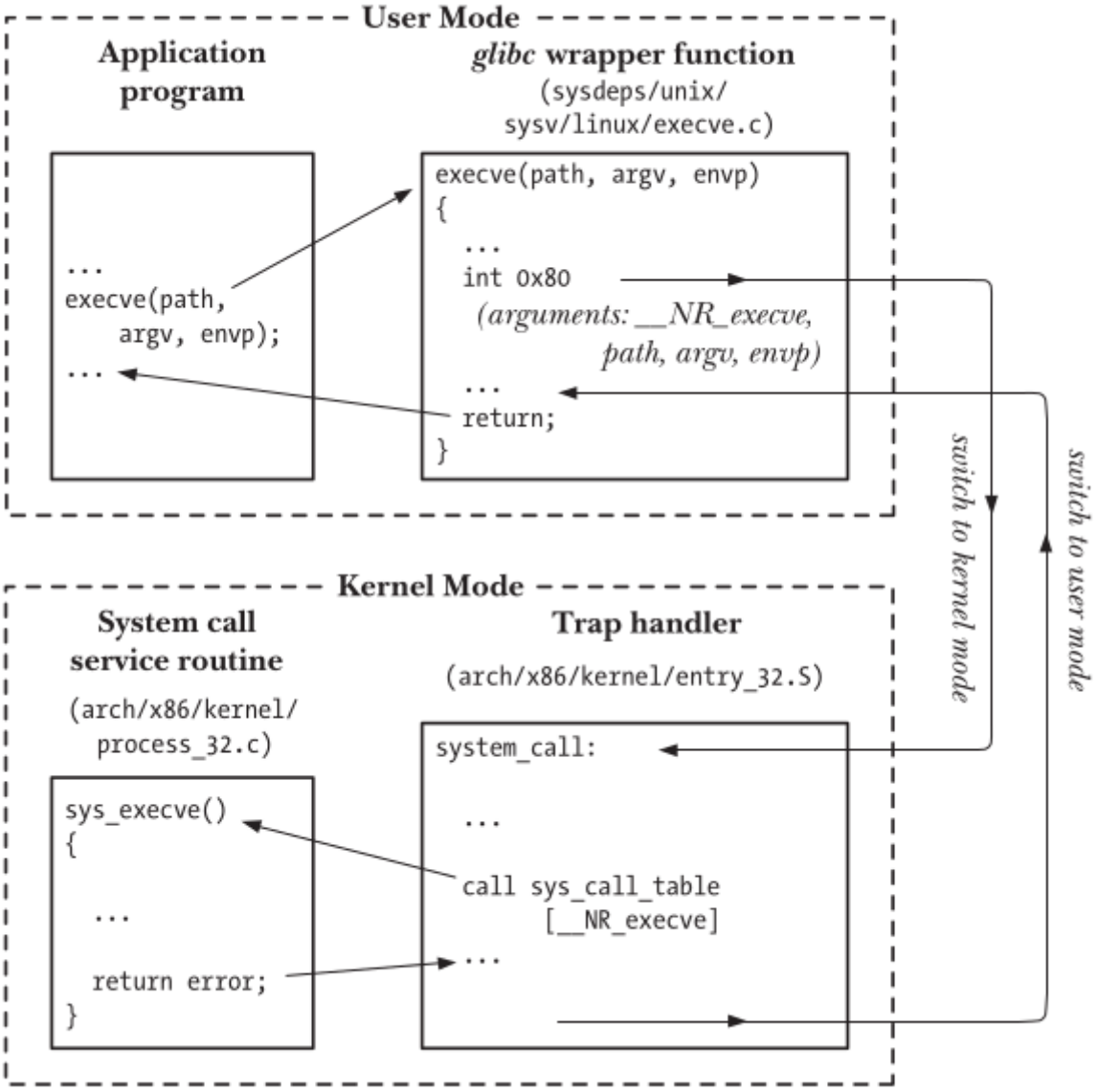
What it is: When you call the write(1, "Hello", 5) function in C, you are in User Space but want the operating system kernel (Kernel Space) to print "Hello" to the hardware screen.

The nature of it: These two memory spaces are separate; the kernel cannot safely and arbitrarily read data directly inside the application's memory. Therefore, the operating system provides a safe mechanism to "copy" or pass parameters (such as string addresses, file sizes, etc.) from your side to the kernel side for processing.

- From a programming perspective, calling a system call looks identical to calling a normal C function (via a wrapper function in the C library), but many complex steps happen underneath.

What it is: When you write code, you simply type open("file.txt", O_RDONLY); and see it return a file descriptor number. It looks no different from writing your own total-calculation function tinh_tong(a, b).

The nature of it: The standard C library (such as glibc) provides you with a wrapper function. The role of this wrapper layer is to hide all the underlying complexity: it automatically places the system call number into a CPU register, issues an interrupt instruction to switch modes, waits for the kernel to finish working, receives the returned result, and smoothly hands it back to you.



1: User Mode Region:

Application program:

- This is your application program.
- When your code calls the execve(path, argv, envp); function, the program is running in user space.

glibc wrapper function:

- Instead of jumping straight into the kernel, the application calls a wrapper function located in the standard C library (glibc).
- This function's task: prepares the parameters, puts the system call number (__NR_execve) into a CPU register, and executes the interrupt instruction int 0x80 (or sysenter). At this point, the transition to kernel mode occurs (switch to kernel mode).

2: Kernel Mode Region (Bottom):

Trap handler (system_call:):

- When the int 0x80 interrupt instruction is executed, the CPU jumps to the kernel's interrupt handling code (arch/x86/kernel/entry_32.S).
- Here, the kernel reads the system call number (__NR_execve), looks it up in the sys_call_table service table to find the corresponding handling function (like looking up an index).

System call service routine (sys_execve()):

- From the sys_call_table, the kernel calls the actual service function of the kernel, sys_execve() (located in arch/x86/kernel/process_32.c).
- This function performs the actual work (e.g., executing a new binary file for the process).
- After finishing, it returns the result (or an error code) back to the trap handler.

3: Execution Flow Return Path:

- After sys_execve() returns the result to the trap handler, the process switches back to user mode (switch to user mode).
- Control returns to the glibc wrapper function; this function checks if there is an error, sets the errno variable, and then returns the result back to the original Application program.

Performance and Overhead:

Because it requires switching back and forth between user mode and kernel mode, performing security checks, and transferring data, a system call incurs significantly higher overhead than a regular C function call.

Note: A system call does not mean a process is always switched to another process or put to sleep. That depends on the specific operation.

- No process switching: When a process calls getppid() (to get the parent process ID) or time() (to get the system time), the process continues to run normally. The kernel only temporarily borrows the CPU to fetch the data and immediately returns it to that exact process, without switching to another process.
- Not always sleeping: Some system calls, like read() when reading data from a keyboard or disk without data ready yet, cause the process to switch to a sleeping state (blocking). Conversely, many system calls finish immediately, and the process continues running in user space without missing a beat.

System calls are useful when you need the kernel to perform an operation.

Requirement	Common System Call
Get process information	`getpid()`

Requirement	Common System Call
Create a process	`fork()`
Open a file	`open()`
Read data	`read()`
Write data	`write()`
Create a socket	`socket()`
Establish a TCP connection	`connect()`
Wait for file descriptor events	`poll()`, `epoll_wait()`

Conclusion on System Calls:

- System calls serve as the foundational interface for User Space to request supported operations from the Kernel.
- However, system calls are not always the most direct or appropriate choice for every communication need. The kernel also provides more specialized interfaces such as ioctl, procfs, and Netlink.

2.2 syscall(2)

Dedicated wrapper functions (e.g., getpid(), open(), read()): These are pre-written functions specific to each individual system call. Inside the code for getpid(), developers have already embedded system call number 39 and the mode-switching instruction (syscall).

General wrapper function (syscall()): This is also provided by the C library, but it does not correspond to any fixed system call. It acts as a universal bridge.

What happens when the C library does not have a wrapper for a new system call?

Suppose Linux introduces a brand-new system call with number 450, but the C library (glibc) version on your machine is old and has not been updated with source code to generate a dedicated wrapper for it.

At this point, if you try to call that new function name directly, the compiler will immediately throw an error because it cannot find the definition.

However, if you use syscall(), the mechanism works as follows:

- You call: syscall(450, parameter_1, parameter_2);
- Library action: The syscall() function itself within the C library does not need to know what number 450 means. Its sole task is to take that number 450, place it directly into a CPU register, and issue the mode-switching instruction (syscall) down to the kernel.

- Kernel Space execution: When the operating system kernel receives number 450 from the CPU, the kernel itself looks up its system table and executes the corresponding code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h> // Contains SYS_getpid definition

int main() {
    // Method 1: Using the standard wrapper function
    pid_t pid_standard = getpid();
    printf("PID retrieved via getpid() wrapper: %d\n", pid_standard);

    // Method 2: Using the general syscall() function to call the number directly
    // On x86_64, SYS_getpid has a value of 39
    long pid_syscall = syscall(SYS_getpid);
    printf("PID retrieved via syscall(): %ld\n", pid_syscall);

    return 0;
}
```

```
Output:

PID retrieved via getpid() wrapper: 30288
PID retrieved via syscall(): 30288
```

The Key Differences Between the Two Approaches

Criteria	Approach 1: Using a Wrapper Function (`getpid()`)	Approach 2: Using the Generic Function (`syscall()`)
Basic Concept	Calls a special-purpose function designed to get the PID, provided by the C library (`glibc`).	Calls a generic function that takes a system call number as its first argument and then passes the remaining arguments.
Internal Optimization	May be optimized by the C library (e.g., caching the PID in some older versions to avoid unnecessary system calls when the process does not change).	Does not provide the same function-specific optimizations. It generally requests the specified system call directly from the kernel.
Purpose	Makes code easier to write and read, hiding low-level details from the programmer.	Provides a general way to invoke a system call when a specific wrapper function is unavailable or when direct access is needed.

Note: All specialized functions in the C library attempt to minimize the actual number of system calls down to the lowest possible level by using buffers, ...

3. IOCTL interface: deep dive + writing a simple C program that issues an ioctl() to a device

ioctl stands for Input/Output Control.

It is an interface that allows a User Space program to send specialized control requests to an object that supports ioctl, typically a hardware device or device driver.

Core ideas:

- Problem. In UNIX "everything is a file", but read()/write() only move bytes. Devices also need operations like setting a serial port's baud rate or reading the terminal size. Adding one syscall per such operation would bloat the kernel.
- Solution. One syscall acts as a common gate. The meaning of request and arg is defined by each driver; the kernel just forwards them.
- Mechanism. The kernel uses the fd to find the driver and calls the driver's ioctl handler (registered in file_operations), which is usually a switch (request). Pointer data is copied with copy_from_user / copy_to_user.
- Request codes are built with the _IO, _IOR, _IOW, _IOWR macros, encoding a driver ID, command number, data direction, and data size, to avoid collisions.

4. proc file system interface: deep dive + writing a simple C program that reads/writes a proc entry

4.1 The Concept of procfs

procfs stands for **process file system**. It is **a virtual filesystem** provided by Linux.

procfs allows User Space to access various types of information, including:

```
Process information.


System status information.
```

```
Kernel-related information.

Networking information.
```

Exmaples

cat /proc/cpuinfo (CPU Information:)

Intel 12th Gen Hybrid Architecture: i5-12450H chip has a total of 8 physical cores (cpu cores : 8), but they are divided into two types:

- Performance-cores (P-cores): High-performance cores supporting Hyper-Threading technology -> 1 physical core runs 2 threads.
- Efficient-cores (E-cores): Power-efficient cores that do not support Hyper-Threading -> 1 physical core runs 1 thread.

cat /proc/meminfo (Memory Information)

Parameter	Value (kB)	Approx. (GB)	Meaning
`MemTotal`	15,984,284	16.37 GB	Total RAM available to Linux
`MemFree`	6,159,160	6.31 GB	RAM that is currently completely unused
`MemAvailable`	10,418,792	10.67 GB	Estimated RAM available for new applications without swapping
`Buffers`	116,660	0.12 GB	Memory used for buffers, mainly for I/O operations
`Cached`	4,192,780	4.29 GB	Memory used for caching, mainly related to file data
`SwapTotal`	4,194,300	4.29 GB	Total swap space
`SwapFree`	4,194,300	4.29 GB	Available swap space

4.2 Why is it called a virtual filesystem?

It is called "virtual" because:

- Zero disk footprint: The files you see (such as /proc/meminfo or /proc/cpuinfo) do not exist on your hard drive at all.
- Generated directly in RAM: When you run a command to read a file, the Linux Kernel immediately fetches information directly from RAM, packages it into text format, and displays it to you in real time.

Case 1: Regular File

For example, you create a file:

```
echo "Hello Linux" > hello.txt
```

=> Data is written to the file and typically stored on the filesystem of the storage device.

When reading it back:

```
cat hello.txt
```

The program reads the data that has been saved in the file.

=> Hello Linux

Case 2: procfs

cat /proc/uptime ()

- The content is not simply static data stored in a regular file on the disk.
- Instead, procfs provides an interface for the Kernel to return the corresponding information on the fly.

first time: cat /proc/uptime => 5958.22 second time: cat /proc/uptime => 6406.86

4.3 writing a simple C program that reads a proc entry

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

#define BUFFER_SIZE 256

int main(void)
{
    int fd;
```

```
    char buffer[BUFFER_SIZE];

    fd = open("/proc/uptime", O_RDONLY);

    if (fd == -1) {
        fprintf(stderr, "open failed: %s\n", strerror(errno));
        return EXIT_FAILURE;
    }

    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);

    if (bytes_read == -1) {
        fprintf(stderr, "read failed: %s\n", strerror(errno));
        close(fd);
        return EXIT_FAILURE;
    }

    buffer[bytes_read] = '\0';

    printf("Content of /proc/uptime:\n");
    printf("%s", buffer);

    if (close(fd) == -1) {
        perror("close");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Output

```
Content of /proc/uptime:
9183.40 103895.96
```

Is procfs a System Call?

No!

- System Call: The interface for User Space to request actions from the Kernel.
- procfs: A virtual filesystem providing entries to access information.

- open() and read(): System calls that programs can use to access those entrie

5. Netlink Socket interface: deep dive + writing a simple C program that opens an AF_NETLINK socket and exchanges a message with the kernel (e.g. NETLINK_ROUTE)

5.1. What is a Netlink Socket?

Netlink is a communication mechanism between user space and kernel space in Linux, using a message-based structure and a socket interface But instead of communicating over the internet, it communicates internally between the application and the operating system kernel.

What is Netlink used for?

```
Typical applications include:

    Retrieving network interface information.

    Retrieving and managing IP address information.

    Receiving notifications about networking-related changes.

    Querying socket information within the kernel via specialized Netlink protocols.
```

Why is Netlink so important?

Previously: Older tools like ifconfig or route used the ioctl() system call to configure networks. This approach had a major drawback: one-way synchronous communication (userspace asks, kernel answers), lack of flexibility, and difficulty handling dynamic network change events.

Asynchronous Notification: The kernel does not just wait to be asked before replying. When a network event occurs (for example, you plug in an Ethernet cable, Wi-Fi drops, or a new IP address is assigned), the kernel actively sends notifications via Netlink to all listening processes in user space. As a result, applications do not need to use resource-heavy "polling" techniques (continuously querying for status).

Multicast Support: A single message from the kernel can be broadcast simultaneously to multiple different applications in user space at the same time.

Extensibility: Netlink supports various "families" for specific purposes:

- NETLINK_ROUTE: Manages routing, IP addresses, link states, and neighbor tables.
- NETLINK_NETFILTER: Interacts with the firewall system (iptables/nftables).
- NETLINK_SOCK_DIAG: Queries information about open sockets.

6. Why Netlink matters for network applications: the standard way Linux userspace networking tools (ip, ss, NetworkManager) talk to the kernel's networking subsystem

ip: Part of the iproute2 tool suite, used to view and manage network configurations on Linux.

ss: Used to inspect, statistics, and analyze open network connections on the system (TCP, UDP, UNIX domain sockets, etc.).

NetworkManager: A network connection management service on many Linux desktop and embedded systems:

- Manages Wi-Fi and Ethernet connections.
- Manages IP address configurations.
- Monitors interface status.

Tool	Role Description	How It Uses Netlink
`ip` (from the `iproute2` package)	Replaces older commands such as `ifconfig` and `route`.	When run `ip addr show`, the tool opens a Netlink socket, sends a request using `RTM_GETADDR` to the kernel, and receives a list of IP addresses.
`ss`	Replaces `netstat` for checking network connections.	Uses `NETLINK_INET_DIAG` to query the kernel for socket information quickly and efficiently.
`NetworkManager` / `systemd-networkd`	Background services that automatically manage network connections on desktop computers and servers.	Continuously listens for Netlink notifications from the kernel to detect network changes, such as an interface coming up or going down

Revision #1
Created 2026-10-19 10:36:30 UTC by Dat Nguyen
Updated 2026-10-19 10:42:21 UTC by Dat Nguyen